SOFTWARE DESIGN & ARCHITECTURE

Quiz 2 - Design Patterns

-CHAPTER 6: CREATIONAL DESIGN PATTERNS

-CHAPTER 7: STRUCTURAL AND BEHAVIORAL DESIGN PATTERNS

Table of Contents

1. Questions (1-10) 2

Question 1 2

Question 2 2

Question 3 2

Question 4 3

Question 5 3

Question 6 3

Question 7 4

Question 8 6

Question 9 7

Question 10 7

2. Answer Key 8

1. Questions (1-10)

Question 1

The following is true about the Singleton design pattern:
(Select ONE or MORE answers)

a. Singletons are known for working well in concurrent environments.
b. Singletons have a private constructor and can have one or more public methods.
c. Although having a private constructor is recommended, Singletons are known to work as well with public constructors.
d. Singletons require a static method with public visibility for instantiation.

Question 2

In software, sometimes your design requires objects to be composed of many parts. In some specific instances, some problems require both objects and their parts to be treated uniformly; therefore these objects can be composed into tree structures to represent part–whole hierarchies. This can be accomplished with the following design pattern:

a. Adapter
b. Facade
c. None of the options
d. Observer

Question 3

You are given the following code:

```
class X { };
class A : public X { };
class B : public X { };
class C : public X { };

class Simplifier {
  X* simplifiedInterface(int option) {
    if( option == 1 ) return new A;
    else if( option == 2) return new B;
    else if( option == 3) return new C;
    else return null;
  }

  X* clone() { return null; }
  static X* getInstance() {
    return new Simplifier;
  }
};
```

What design pattern is employed by the class `Simplifier`?

a. None of the above
b. Prototype
c. Singleton
d. Facade

Question 4

The following design pattern is used to separate the construction process of a complex object from its representation so that the same construction process can create different representations:

a. Abstract Factory
b. Builder
c. None of the above
d. Factory Method

Question 5

The following pattern can be used to provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

a. Composite
b. Iterator
c. None of the above
d. Observer

Question 6

Consider the following C++ code:

```
class Person {
public:
  virtual void speak() = 0;
  Person* clone() { /* intentionally left blank. */}
};

// FYI, Persona is the spanish translation of Person.
class Persona {

  // FYI, hablar is the spanish translation of speak.
  virtual void hablar () = 0;

  Persona* clone() { /* intentionally left blank. */}
};

class X : public Person {
public:
  X(Persona* persona) {
    _persona = persona;
  }

  void speak() {
    _persona->hablar();
  }
private:
  Persona* _persona;
};
```

The specific design pattern(s) employed in this code is/are:

a. Adapter and Prototype
b. None of the above
c. Adapter
d. Composite

Question 7

You are given the following classes:

```
class Transmitter {

public:
  virtual void transmit() = 0;
  void clone() { /* intentionally left blank. */}
};

class Receiver {
public:
  virtual void receive() = 0;
  void clone() { /* intentionally left blank. */}
};

class FastTransmitter : public Transmitter {
  // code here
};
class SlowTransmitter : public Transmitter {
  // code here
};
class FastReceiver : public Receiver {
  // code here
};
class SlowReceiver : public Receiver {
  // code here.
};
class RadioBuilder {
public:
  Transmitter* createTransmitter() = 0;
  Receiver* createReceiver() = 0;
  void clone() { /* intentionally left blank. */}
};

class FastRadioBuilder : public RadioBuilder {
public:
  Transmitter* createTransmitter() {
    return new FastTransmitter;
  }

  Receiver* createReceiver() {
    return new FastReceiver;
  }
};

class SlowRadioBuilder : public RadioBuilder {
public:
Transmitter* createTransmitter() {
    return new SlowTransmitter;
  }

  Receiver* createReceiver() {
```

```
    return new SlowReceiver;
  }
};

class Radio {
  // constructor.
  Radio(RadioBuilder* builder) {
    _transmitter = builder->createTransmitter();
    _receiver = builder->createReceiver();
  }
  Radio* createRadio(void) { //... };
private:
  Transmitter* _transmitter;
  Receiver* _receiver;
};
```

The design pattern that best describes the relationships between these classes is:

a. Builder
b. Abstract Factory
c. Factory Method
d. Prototype

Question 8

Consider the following C++ code:

```
class Character {
public:
// Public method.
Character* clone() { return new Character; }
// Public method.
Character* createCharacter() { return new Character; }
};
class NewCharacter : public Character { ... };
// The program's main method.
void main() {
  Character* pCharacter = new NewCharacter;
  // use the pCharacter to create characters and clones.
}
```

The specific design pattern(s) employed in this code is/are:

a. Factory Method
b. Factory Method and Prototype
c. Prototype
d. None of the above

Question 9

A Model-View-Controller architectural pattern is used for a system that has multiple views of the same data, so that when the data changes, all views need to update. Therefore, the detailed design must support a system that stores references to each view, and upon change, iterates through them to notify them of changes. The appropriate design pattern used to provide this "publish-subscribe" functionality during detailed design is:

a. Iterator
b. Observer
c. Adapter
d. All of the above

Question 10

Consider a user interface framework that supports multiple look-and-feel standards. Different look-and-feels (i.e., themes) define different appearances and behaviors for user interface "widgets" like command buttons, scroll bars, windows, list boxes, check boxes, etc. The framework currently supports two themes, the "Windows Classic Theme" and the "Windows 7 Theme," but others may be added in the future. The Windows 7 theme is a duplicate of the Microsoft Windows 7 theme; that is, when using the Windows 7 theme, all user interface widgets look and behave according to Windows 7; the same applies to the Windows Classic theme. The specific design pattern(s) that is/are most appropriate for designing this feature into the user interface framework is/are:

a. Factory Method and Prototype
b. Creational Factory
c. Abstract Factory and Prototype
d. None of the above

2. Answer Key

Question	Correct Answer(s)
Q1	b, d
Q2	c
Q3	a
Q4	b
Q5	b
Q6	c
Q7	b
Q8	d
Q9	b
Q10	d